

RoPE

Rotating vectors to encode order · video 2:35

This lesson answers one question: how do modern models encode position without a learned table? **RoPE** (rotary position embedding) **rotates** each query and key by an angle proportional to its position. Because a dot product depends only on the angle between two vectors, the attention score then depends only on the **relative distance** between tokens. A head's vector is split into pairs that rotate at many different speeds, from a turn every 6 tokens to one every 4 million.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Position as a rotation	▶ 0:08-0:42	2	☐
2. Why the score depends only on distance	▶ 0:42-1:27	3	☐
3. Many speeds, like clock hands	▶ 1:27-2:04	2	☐
4. Stretchable, and four lines of code	▶ 2:04-2:35	2	☐

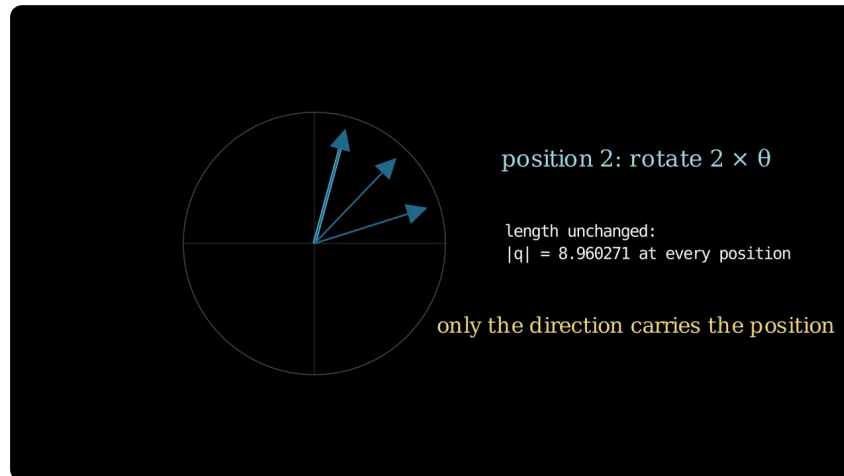
Before you start: Deep Dive episode 4 (why attention needs position) and Foundations F02 and F09 (dot product, sine, cosine and rotations). Code: `code/d05_rope` (only a config download).

CONCEPT 1 OF 4

Position as a rotation

▶ Video 0:08-0:42

GPT-2's learned table stops at 1,024 positions. Most modern models (Llama, Mistral, Qwen) use **rotary position embedding**, RoPE, instead. Picture a query as an arrow with two numbers. At position 0, leave it alone; at position 1, rotate it by a small angle θ ; at position 2, by 2θ ; and so on. The arrow keeps its length ($|q| = 8.960271$ before and after); only its direction carries the position.



▶ 0:40 Rotate the vector by θ per position: the length stays the same; only the direction carries the position.

KEY IDEA

RoPE does not add a position vector: it turns the query and key by position-dependent angles.

Check 1: Position as a rotation

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

$\theta = 30^\circ$ per position. By how many degrees is a query at position 5 rotated?

1.2 TRUE OR FALSE

“Rotating a vector by RoPE changes its length.”

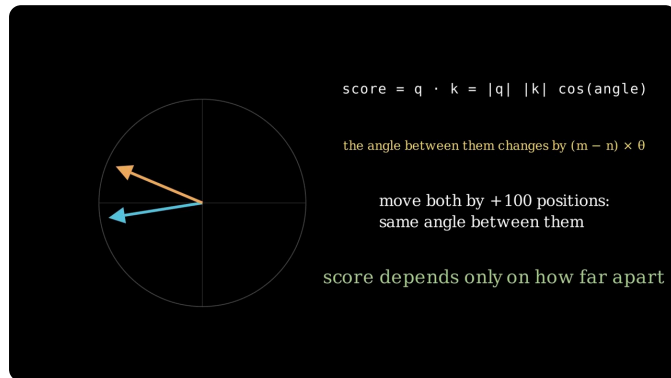
True ☐ False ☐

CONCEPT 2 OF 4

Why the score depends only on distance

▶ Video 0:42-1:27

Attention scores are **dot products**, and a dot product depends on the **angle between** two vectors: $q \cdot k = |q| |k| \cos(\text{angle})$. Rotate the query by its position m and the key by its position n ; the angle between them changes by $(m - n) \theta$. So the score depends only on how far apart the two tokens are, not on where they are.



▶ 1:02 Rotate the query by its position m and the key by n : the angle between them changes by $(m - n)\theta$.

random query and key, head size 64: $\text{score} = \text{rope}(q, m) \cdot \text{rope}(k, n)$

$m =$	5	$n =$	2	$m - n =$	3	+5.924725
$m =$	105	$n =$	102	$m - n =$	3	+5.924725
$m =$	10,005	$n =$	10,002	$m - n =$	3	+5.924725
$m =$	5	$n =$	4	$m - n =$	1	+8.349663
$m =$	105	$n =$	104	$m - n =$	1	+8.349663

▶ 1:26 Same distance, same score, wherever the pair is: +5.924725 for $m - n = 3$.

Checked with a random query and key of head size 64: positions (5, 2), (105, 102) and (10,005, 10,002) all give **+5.924725**; a distance of 1 gives a different score, +8.349663, again the same everywhere.

KEY IDEA

$\text{rope}(q, m) \cdot \text{rope}(k, n)$ is a function of q , k and $m - n$ only: RoPE turns absolute rotations into relative attention.

Check 2: Why the score depends only on distance

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

In the code's example, what is the score for $m = 1,000,003$ and $n = 1,000,000$?

2.2 MULTIPLE CHOICE

Which pair has the same score as (53, 3)?

- A. (53, 4)
- B. (1053, 1003)
- C. (103, 3)
- D. (3, 53)

2.3 SHORT ANSWER

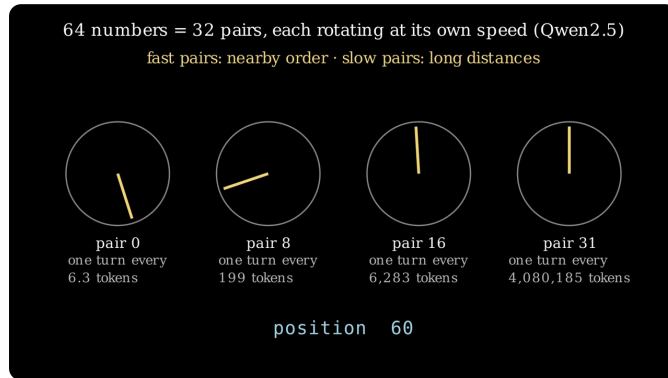
Why does the value vector not need to be rotated?

CONCEPT 3 OF 4

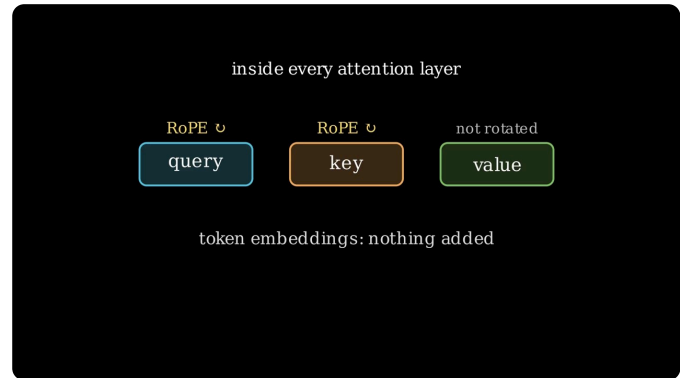
Many speeds, like clock hands

▶ Video 1:27-2:04

A head of 64 numbers makes **32 pairs**, each rotating at its own speed: frequency = $\text{base}^{-2i/64}$ for pair i . In Qwen2.5 (base 1,000,000) the fastest pair makes a full turn every **6.3** tokens, pair 8 every 199, pair 16 every 6,283, and the slowest every **4,080,185** tokens. Fast pairs track nearby order; slow pairs measure long distances.



▶ 1:49 Four of Qwen2.5's 32 pairs after 60 positions: the fast pair has turned many times; the slowest has barely moved.



▶ 2:02 RoPE rotates queries and keys only; nothing is added to the token embeddings.

RoPE is applied inside every attention layer, to the queries and keys only. Nothing is added to the token embeddings, and the values are not rotated: position only shapes who attends to whom.

KEY IDEA

Many rotation speeds give the model both fine and coarse position information, like the second, minute and hour hands of a clock.

Check 3: Many speeds, like clock hands

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Pair 16 rotates at $\text{base}^{-32/64}$ radians per token with base 1,000,000. What is that, and how many tokens per full turn (2π radians)?

3.2 SHORT ANSWER

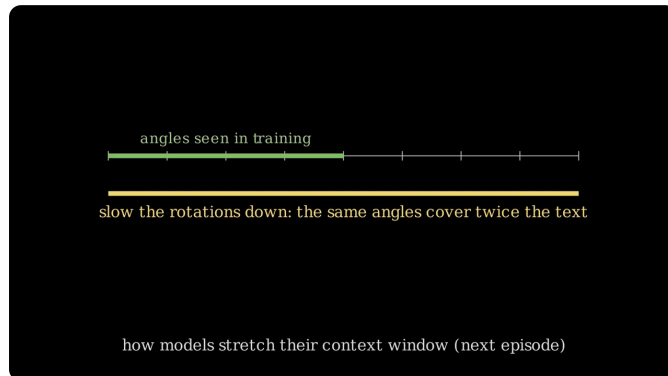
Why are the fastest pairs alone not enough to encode position?

CONCEPT 4 OF 4

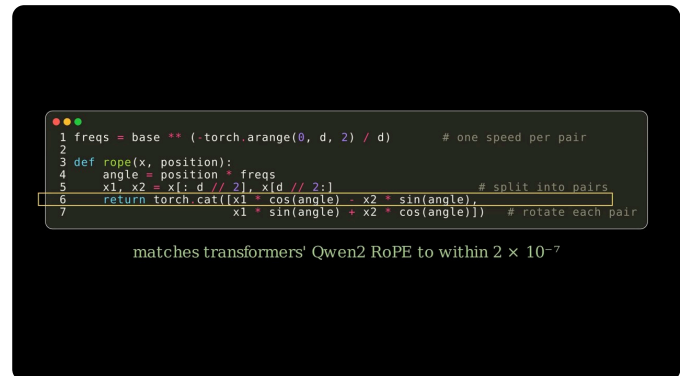
Stretchable, and four lines of code

▶ Video 2:04-2:35

Because position is just an angle, it can be **rescaled**: slow the rotations down, and the same angles cover a longer text. That is how models stretch their context window (next episode).



▶ 2:14 Slow the rotations down, and the same angles cover a longer text.



▶ 2:28 RoPE in four lines; it matches transformers' Qwen2 implementation to within 2×10^{-7} .

In code, RoPE is four lines: compute the angles, split the vector into two halves, and rotate each pair with a cosine and a sine. The episode's version matches the one in Hugging Face's Qwen2 code to within two ten-millionths (float rounding).

KEY IDEA

RoPE has no learned parameters: it is pure geometry, which is why it can be adjusted after training.

Check 4: Stretchable, and four lines of code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 TRUE OR FALSE

“RoPE adds a learned table of parameters to the model, like GPT-2's position embeddings.”

True ☐ False ☐

4.2 HANDS-ON CODE

Try it yourself. Using `rope`, `q` and `k` from `rope.py`, print the score for each pair below. Which scores are equal, and why?

```
for m, n in [(7, 4), (53, 3), (1053, 1003), (1_000_003, 1_000_000)]:  
    print(m, n, torch.dot(rope(q, m), rope(k, n)).item())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Position as a rotation · rewatch ▶ 0:08-0:42

1.1 150°. $5 \times 30^\circ = 150^\circ$.

1.2 False. A rotation preserves length: $|\text{rope}(q, 7)| = |q| = 8.960271$ in the code.

2. Why the score depends only on distance · rewatch ▶ 0:42-1:27

2.1 +5.924725. $m - n = 3$, the same distance as (5, 2) (checked by running it).

2.2 B. Both are 50 apart; the code gives +11.24301 for each.

2.3 Position only needs to affect who attends to whom, which is decided by the query-key scores; the values are what is mixed once the weights are set. RoPE is applied to queries and keys only, inside every attention layer.

3. Many speeds, like clock hands · rewatch ▶ 1:27-2:04

3.1 0.001 rad/token; about 6,283 tokens. $1,000,000^{-0.5} = 0.001$; $2\pi / 0.001 \approx 6,283.2$.

3.2 They wrap around every few tokens, so positions 0 and 6 look almost the same to them; slow pairs are needed to tell far-apart positions apart. Like a second hand: it repeats every minute, so you also need the minute and hour hands.

4. Stretchable, and four lines of code · rewatch ▶ 2:04-2:35

4.1 False. The rotation angles are computed from a formula (positions $\times$ fixed frequencies); nothing is learned.

4.2 (7, 4) and (1,000,003, 1,000,000) both give +5.924725 (distance 3); (53, 3) and (1053, 1003) both give +11.24301 (distance 50). Only $m - n$ matters, even a million positions away.